

VisionEngine

Betrachten Sie die UI wie ein Nutzer - Computer Vision kombiniert mit LLM-basierter Analyse und Navigation.

Zusammenfassung

VisionEngine ist ein entkoppeltes Go-Toolkit, das klassische Computer Vision mit LLM-basierter Bildverarbeitung verbindet, um Benutzeroberflächen zu analysieren, UI-Elemente und visuelle Fehler zu erkennen sowie Navigationsgraphen von Bildschirmübergängen in Apps zu erstellen - mit austauschbaren Vision-Backends mehrerer Anbieter und OpenCV, das hinter einem Build-Tag gekapselt ist.

Kurzbeschreibung

Ein wiederverwendbares Go-Modul für UI-Analyse und die Erstellung von Navigationsgraphen. Es bietet eine Analyseschicht (UI-Elemente, Bildschirmvergleiche, visuelle Fehler), einen Navigationsgraphen mit Breitensuche (BFS) und Exportformaten wie DOT, JSON und Mermaid sowie LLM-Vision-Adapter für GPT-4o, Claude, Gemini, Qwen-VL und weitere.

Ausführliche Beschreibung

Die meisten UI-Testautomatisierungen sind praktisch blind. Sie greifen auf Barrierefreiheitsbäume und DOM-Selektoren zurück - eine maschinelle Interpretation der Oberfläche - und übersehen alles, was ein Mensch tatsächlich wahrnimmt: ob ein Button sichtbar gerendert ist, ob das Layout beschädigt wurde oder ob der erreichte Bildschirm der erwartete ist. VisionEngine schließt diese Lücke, indem es der Automatisierung echte Wahrnehmung verleiht - die Fähigkeit, eine UI zu betrachten und sie so zu verstehen, wie es ein Mensch tun würde. Das System ist in vier kooperierende Schichten unterteilt, die von Rohpixeln bis zum Verständnis der gesamten Anwendung aufbauen.

Die **Analyseschicht** definiert den stabilen Vertrag – Schnittstellen (Analyzer, VideoProcessor) und Werttypen (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) mit einer Referenzimplementierung (StubAnalyzer) – sodass Nutzer Elemente erkennen, Bildschirme vergleichen und visuelle Fehler anhand eines stabilen Vertrags aufdecken können, der sich nicht unter ihnen verändert.

Der **Navigationsgraph** erweitert die Perspektive von einem einzelnen Bildschirm auf die gesamte Anwendung und modelliert sie als gerichteten Graphen von Bildschirmübergängen mit Breitensuche (BFS) und drei Export-Backends (DOT, JSON, Mermaid). Dadurch kann die Automatisierung nicht nur einen Bildschirm erkennen, sondern auch einen Weg zu jedem anderen planen – und wird mit Stresstests, Automatisierungs-, Integrations- und Sicherheitstest-Suiten ausgeliefert, die dies belegen.

Die **LLM-Vision-Schicht** fügt moderne multimodale Analyse hinzu: eine VisionProvider-Schnittstelle mit Adaptionen für OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica und Ollama, die über eine FallbackChain zusammengesetzt werden. So führt ein fehlerhafter, ratenlimitierter oder schwacher Anbieter nicht zum Abbruch des gesamten Prozesses, sondern wird elegant an den nächsten übergeben.

Eine **Konfigurationsschicht** übernimmt das Laden und Validieren von Umgebungsvariablen, wobei alle nutzerorientierten Fehlermeldungen über den `i18n.Translator` geleitet werden.

Der entscheidende Faktor, der diese Lösung tatsächlich einsatzfähig macht, ist die Option, die schwere native Abhängigkeit wegzulassen. Die OpenCV-Bindings sind hinter einem Build-Tag (`-tags vision`) verborgen, und der Standard-Build liefert Platzhalter aus – sodass das gesamte Modul auf jedem Go-Host ab Version 1.25 kompiliert, getestet und ausgeführt werden kann, ohne dass eine OpenCV-Toolchain erforderlich ist. Die native Komponente wird erst eingebunden, wenn ein Nutzer sie explizit aktiviert. Dadurch lässt sich VisionEngine problemlos in einen einfachen CI-Runner integrieren, ohne ein spezielles Image zu benötigen.

Vollständig entkoppelt gemäß der Vorgabe (CONST-051(B)) wird es von Nutzern – insbesondere HelixQA – als gleichberechtigtes Submodul im Code integriert und verleiht UI-Tests mit datenbasierter Analyse echte „Augen“.

Warum wir es entwickelt haben

UI-Testautomatisierung, die sich ausschließlich auf Accessibility-Trees oder Selektoren verlässt, verfehlt, was der Nutzer tatsächlich sieht. VisionEngine fügt echte visuelle Intelligenz hinzu – Elementerkennung, Bildschirmvergleiche und LLM-basierte Schlussfolgerungen – sowie eine navigierbare Karte der App-Bildschirme, sodass Automatisierung eine Benutzeroberfläche nicht nur wahrnehmen, sondern auch durch sie navigieren kann.

Warum es bahnbrechend ist

Es vereint zwei normalerweise unvereinbare Ansätze – schnelle, deterministische klassische Computer Vision und flexible, semantische LLM-Vision – hinter einer einzigen Schnittstelle mit einer Fallback-Kette. So erhält der Anwender die Präzision des einen und die Schlussfolgerungsfähigkeit des anderen, ohne sich entscheiden zu müssen. Und indem OpenCV strikt optional bleibt, entfällt die übliche Belastung durch diese Leistung: Jedes Go-Projekt kann echte UI-Wahrnehmung nutzen, ohne eine native Vision-Toolchain in seinen Build zu integrieren.

Was innovativ ist

- **Duale Wahrnehmung:** klassische CV (OpenCV/GoCV) plus Multi-Provider-LLM-Vision mit Fallback-Kette.
- **Navigationsgraph** mit Breitensuche (BFS) für Pfadfindung und Exportformaten wie DOT, JSON und Mermaid.
- **Build-Tag-gesteuertes OpenCV**, sodass das Modul ohne native Abhängigkeiten kompilier- und testbar bleibt.
- **Vollständig entkoppeltes, i18n-integriertes Submodul** mit einheitlicher Codebasis (verwendet von HelixQA).

Herausforderungen & Lösungen

- **Probleme durch schwere native Abhängigkeiten:** gelöst durch -tags vision-Gating und Standard-Stubs, sodass CI-Systeme und Hosts ohne OpenCV weiterhin kompilieren und testen können.
- **Unzuverlässigkeit von Vision-Providern:** gelöst durch ein VisionProvider-Interface und einen FallbackChain-Kompositor.
- **Komplexe App-Flows abbilden:** gelöst durch einen gerichteten Navigationsgraphen mit BFS-Pfadfindung und Export in mehrere Formate.
- **Kopplung:** gelöst durch CONST-051(B)-Entkopplung und die i18n-Übersetzungsschicht.

Technologie-Stack (Begründung & Umsetzung)

- **Go (1.25+)** - Modulkern und alle vier Schichten.
- **GoCV / OpenCV** - klassische Computer Vision, per Build-Tag gesteuert.
- **LLM-Vision-Provider (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** - multimodale UI-Analyse über Adapter.
- **Graphenalgorithmen (BFS)** - Pfadfindung in der Navigation.
- **DOT / JSON / Mermaid-Exporter** - Visualisierung des Navigationsgraphen.
- **i18n-Übersetzer** - entkoppelte, nutzerorientierte Texte.